POO

Programação Orientada a Objetos

Gerson Trindade

Classes abstratas e Interfaces

Classes Abstratas x Interfaces

Com o início do paradigma de desenvolvimento Orientado a Objetos, alguns termos se tornam comuns no mundo de programadores e analistas de sistemas. *Classes*, *herança*, *polimorfismo* são alguns destes termos.

Mas em se tratando de Classes, algumas dúvidas quanto às diferenças entre classes do tipo Abstratas e Interfaces ainda persistem.

São comuns os questionamentos: Quando devo usar uma classe abstrata? Quando devo usar uma Interface? Devo usar as duas?

Classes Abstratas x Interfaces

Na verdade, uma ***Classe Abstrata*** sem qualquer implementação, tem o aspecto parecido com uma ***Interface***. Mas ambas possuem várias diferenças e similaridades entre si. Pensando neste tipo de dúvida, veremos neste tópico algumas questões sobre elas.

Classes Abstratas x Interfaces

Interfaces

- Uma interface não é considerada uma Classe e sim uma Entidade.
- Não possui implementação, apenas assinatura, ou seja, apenas a definição dos seus métodos sem o corpo.
- Todos os métodos são abstratos.
- Seus métodos são implicitamente Públicos e Abstratos.
- Não há como fazer uma instância de uma Interface e nem como criar um Construtor.
- Funcionam como um tipo de "contrato", onde são especificados os atributos, métodos e funções que as classes que implementem essa interface são obrigadas a implementar.
- Já que C# não suporta Heranças Múltiplas, as Interfaces são usadas para implementá-las.

Classes Abstratas x Interfaces

Classes Abstratas

- As classes abstratas devem conter pelo menos um método abstrato, que não tem corpo.
- É um tipo especial de classe que não há como criar instâncias dela.
- É usada apenas para ser herdada, funciona como uma super classe.
- Uma grande vantagem é que força a hierarquia para todas as sub-classes.
- É um tipo de contrato que faz com que as sub-classes contemplem as mesmas hierarquias e/ou padrões.

Classes Abstratas x Interfaces

Classes Abstratas

As **classes** são usadas quando você precisa de uma implementação concreta e direta, e as **interfaces** são úteis para definir comportamentos comuns entre várias classes sem impor uma relação de herança direta.

Característica	Classe	Interface
Implementação de Métodos	Métodos concretos e abstratos	Apenas métodos abstratos (com exceções para `default` e `static`)
Herança	Herança simples (uma única classe)	Implementação múltipla (múltiplas interfaces)
Modificadores de Acesso	`public`, `private`, `protected`	Métodos são `public` por padrão (antes do Java 9)
Instanciação	Pode ser instanciada	Não pode ser instanciada diretamente
Construtores	Pode ter construtores	Não tem construtores

Pilares da POO

Pilares da Orientação a Objetos

Sem sombra de dúvidas o paradigma da Orientação a Objetos trouxe um ganho significativo na qualidade da produção de software.

Muito destes benefícios são em decorrência de 4 pilares, que quando colocados em prática, facilitam a vida do programador e proporcionam um código de qualidade.

Os pilares da Orientação a Objetos são: ***Abstração***, ***Encapsulamento***, ***Herança*** e ***Polimorfismo***.

Pilares da POO

Abstração

Conceito de Abstração

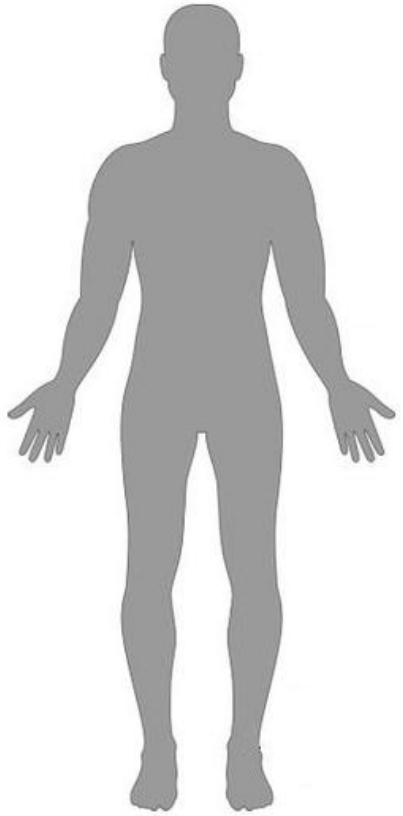
A Abstração é a habilidade e a capacidade de se modelar conceitos, entidades, elementos, problemas e características do mundo real, de um domínio do problema em questão, levando-se em conta apenas os detalhes importantes para a resolução do problema e desprezando coisas que não têm importância no contexto.

Conceito de Abstração

Quando consideramos por exemplo a classe "**Pessoa**", sabemos que cada indivíduo dessa classe tem muitos atributos que diferem de individuo para individuo, porém, quando queremos definir uma classe "**Alunos**", que também são **Pessoas**, por exemplo, não são todos os atributos que precisamos levar em consideração. Assim fazemos uma abstração somente dos atributos relevantes da classe **Pessoa** para definir a classe **Aluno**.

Esse conceito para muitos autores de **POO** é o principal pilar da ***Orientação a Objetos***.

Classe: Pessoa



Classe: Aluno

Classe: Professor

Pilares da POO

Encapsulamento

Conceito de Encapsulamento

Na Orientação a Objetos, encapsulamento é o mecanismo utilizado para disponibilizar métodos que operam sobre os dados e que protegem o acesso direto indevido aos atributos de uma instância fora da classe onde estes foram declarados.

Esta proteção consiste em se usar modificadores de acesso mais restritivos sobre os atributos definidos na classe e fornecendo métodos que alteram os valores destes atributos de alguma forma.

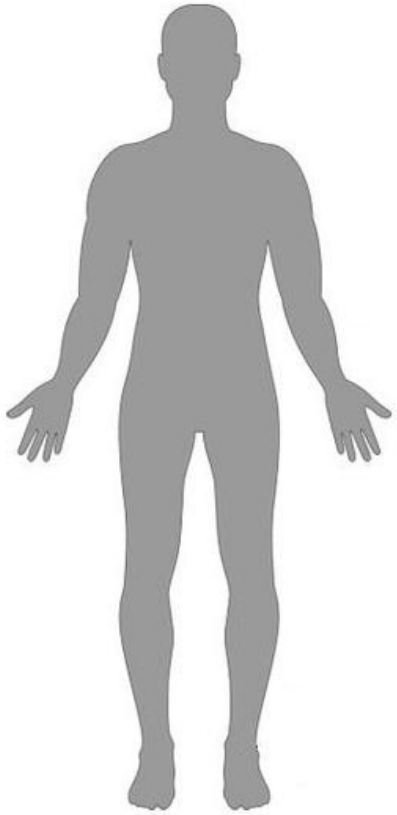
A ideia do encapsulamento é também fechar o sistema, de forma que quem for utilizar os métodos e atributos saiba para que eles servem, e não o que eles fazem ou o que é feito com eles internamente.

Conceito de Encapsulamento

O encapsulamento ajuda a prevenir o problema de interferência externa indevida sobre os dados de um objeto, como objetos que possam alterar os dados de outros objetos indevidamente.

Um exemplo deste problema pode ser o saldo da conta bancária. O saldo certamente não pode ser alterado ou manipulado diretamente, mas sim através de métodos adequados para isso, como métodos que fazem lançamentos de débitos e créditos.

Classe: Pessoa



Classe: Aluno

Classe: Professor

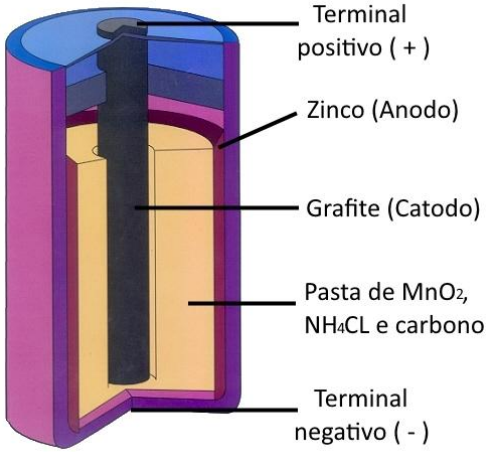
Conceito de Encapsulamento

Vamos utilizar um exemplo natural para explicar a importância do ***Encapsulamento***.

Considere, por exemplo, uma pilha. Uma pilha é uma cápsula, e quando nós utilizamos o exemplo do uso do controle remoto, falamos que um objeto pode conter outros objetos. A pilha é um objeto utilizado pelo controle remoto que também é um objeto.

Você já se perguntou porque que a pilha é uma cápsula ?

Qual o objetivo dos componentes internos da pilha estarem encapsulados ?



Conceito de Encapsulamento

Respondendo à duas perguntas anteriores, podemos dizer que um dos objetivos da pilha estar encapsulada é de ***proteção***. Você ficará protegido contra os elementos químicos contidos na pilha e, por sua vez, a estrutura da pilha estará protegida contra danos que você, como usuário, pode causar ao objeto.

Um outro objetivo da pilha estar encapsulada é para garantir um ***padrão*** de conexão ou encaixe do objeto.

Conceito de Encapsulamento

Ainda podemos considerar que a cápsula torna irrelevante ao utilizador do objeto a maneira como este objeto funciona internamente.

Por exemplo, podemos substituir uma pilha comum por uma alcalina de mesma forma, ou até mesmo por um modelo recarregável, sem prejuízo no uso do objeto.

Conceito de Encapsulamento

Essa analogia com a pilha podemos transferir para os conceitos de encapsulamento em programação orientada a objetos, ou seja, um sistema bem encapsulado apresenta uma proteção do código interno da classe, bem como um padrão de utilização. Lembre-se que a orientação a objeto tem foco na funcionalidade e não na implementação.

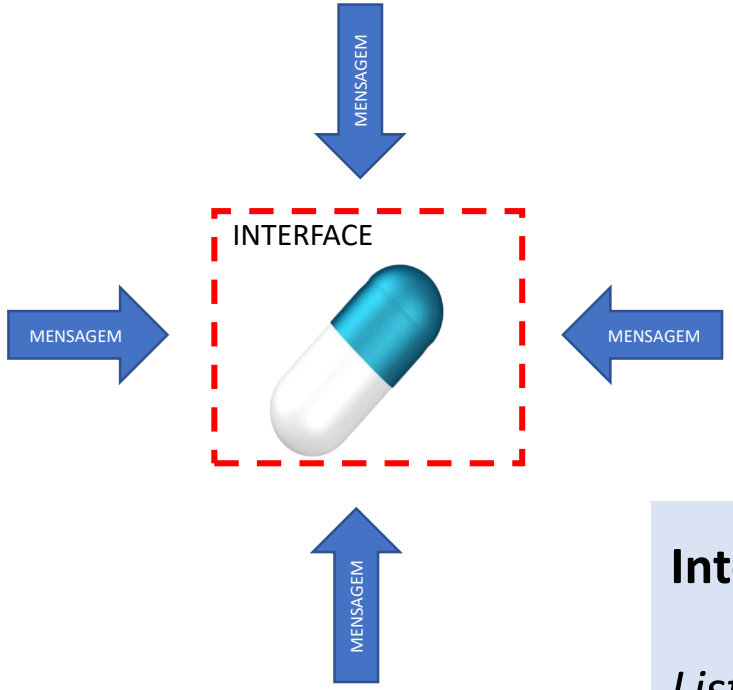
Assim, após o encapsulamento temos disponíveis apenas a interface de uso do objeto, por exemplo, no caso da pilha a interface de uso são os seus polos de conexão.

Para a POO, encapsular significa:

> ***Ocultar partes independentes da implementação, permitindo construir partes invisíveis ao mundo exterior.***

Conceito de Encapsulamento

Desse modo, o utilizador não tem acesso aos elementos internos da cápsula, e a única forma de conversar com a cápsula é por meio de mensagens. Essas mensagens representam a troca de informação entre o mundo externo e os elementos internos da cápsula.



Como foi dito antes o encapsulamento disponibiliza uma interface de uso.

Um objeto bem encapsulado, possui uma interface bem definida.

Interface

Lista de serviços fornecidos por um componente. É o contato com o mundo exterior, que define o que pode ser feito com um objeto dessa classe.

Conceito de Encapsulamento

Encapsular ***não é obrigatório***, mas é uma boa prática para produzir ***Classes*** mais eficientes.

Vantagens de se Encapsular

1. ***Tornar mudanças invisíveis.***
2. ***Facilitar a reutilização do código.***
3. ***Reduzir efeitos colaterais.***

Conceito de Encapsulamento

Implementando o encapsulamento

Para exemplificar a implementação do Encapsulamento vamos retomar o exemplo do "***Controle remoto***" visto em aulas anteriores.

Considere o objeto ***controle remoto*** simplificado mostrado na figura abaixo.

Sabemos que o corpo plástico do controle remoto funciona como uma cápsula para esconder os elementos internos do controle, pois suas implementações internas e forma de funcionamento são irrelevantes para o uso das funções disponíveis ao usuário.

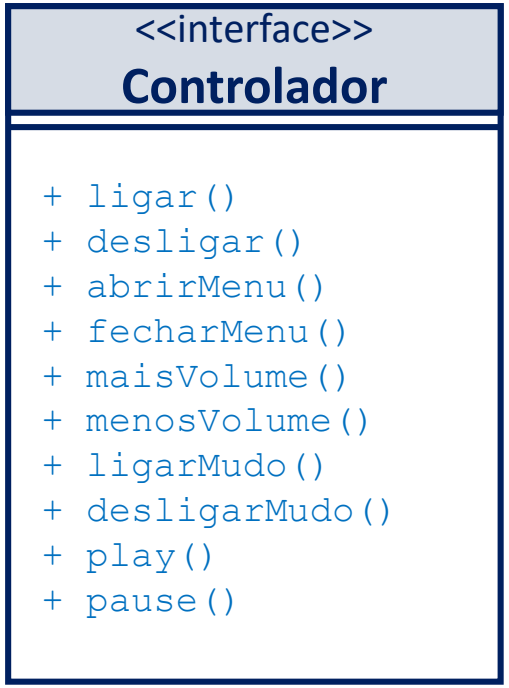
Podemos então dizer, que o conjunto de botões e elementos externos do controle remoto constituem a interface de uso disponível ao utilizador do controle.

Conceito de Encapsulamento

Implementando o encapsulamento

Para começarmos a criar o encapsulamento vamos criar a ***interface***, e para isso vamos utilizar a sintaxe da **UML** novamente.

Uma ***interface*** na **UML** é representada semelhante a uma Classe com a diferença que ela não possui atributos, somente métodos, conforme o diagrama abaixo.



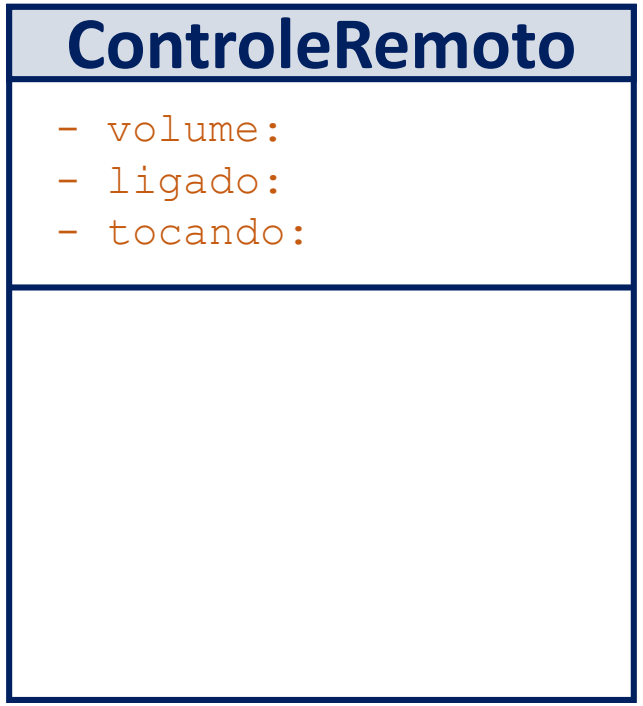
No nosso exemplo vamos criar a ***interface*** **Controlador**, ou seja, todo controle remoto tem quase sempre a mesma interface Controlador.

Os métodos colocados na interface são ***abstratos***, pois não serão desenvolvidos na interface. Eles apenas indicarão o que vai existir na interface do objeto.

Conceito de Encapsulamento

Implementando o encapsulamento

Vamos agora definir a Classe ***ControleRemoto***. Essa classe vai ter os seguintes atributos: volume, ligado, tocando.

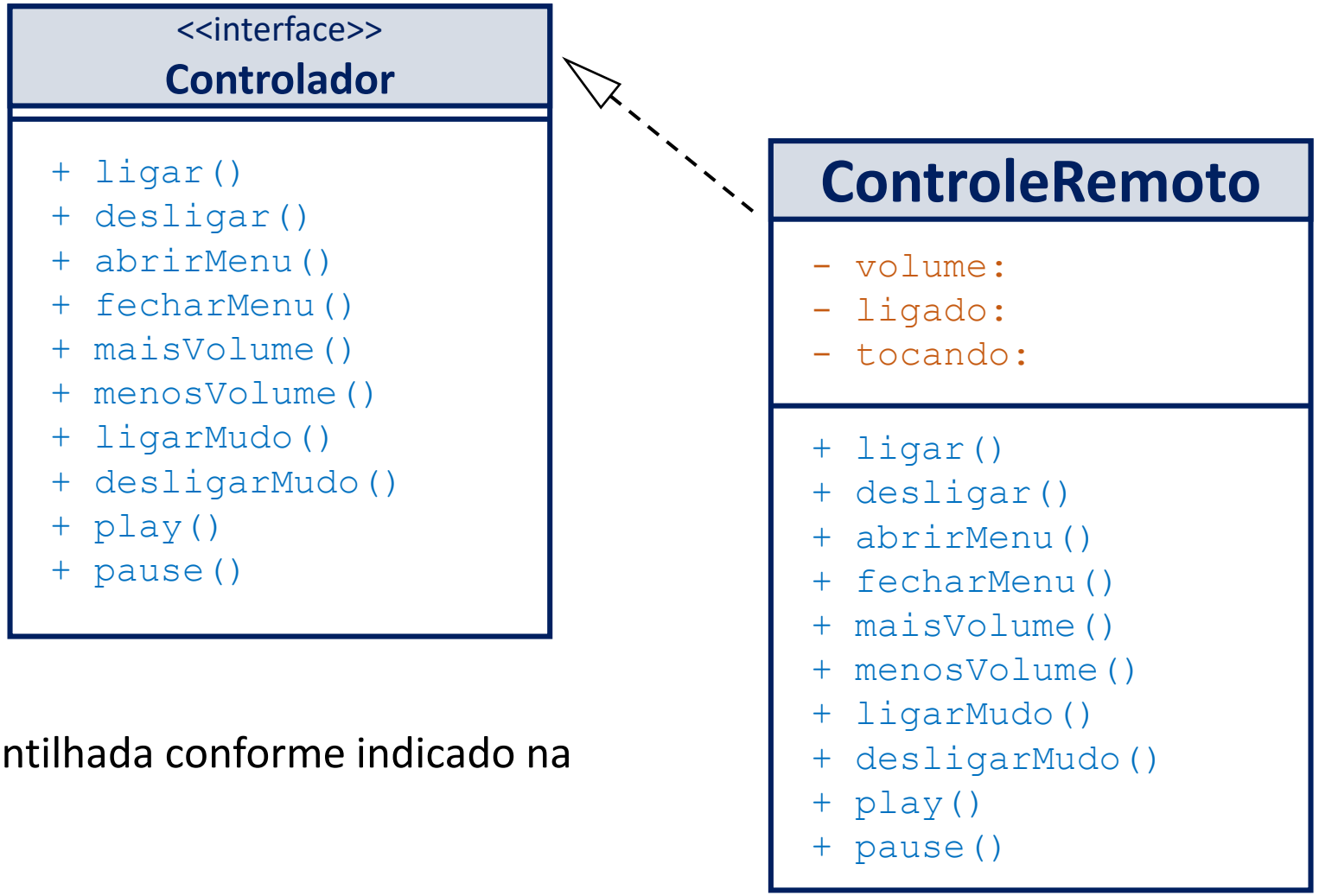


Quando encapsulamos, a primeira coisa a se fazer e tornar todos os atributos ***privados***, ou protegidos, em alguns casos, mas nunca públicos.

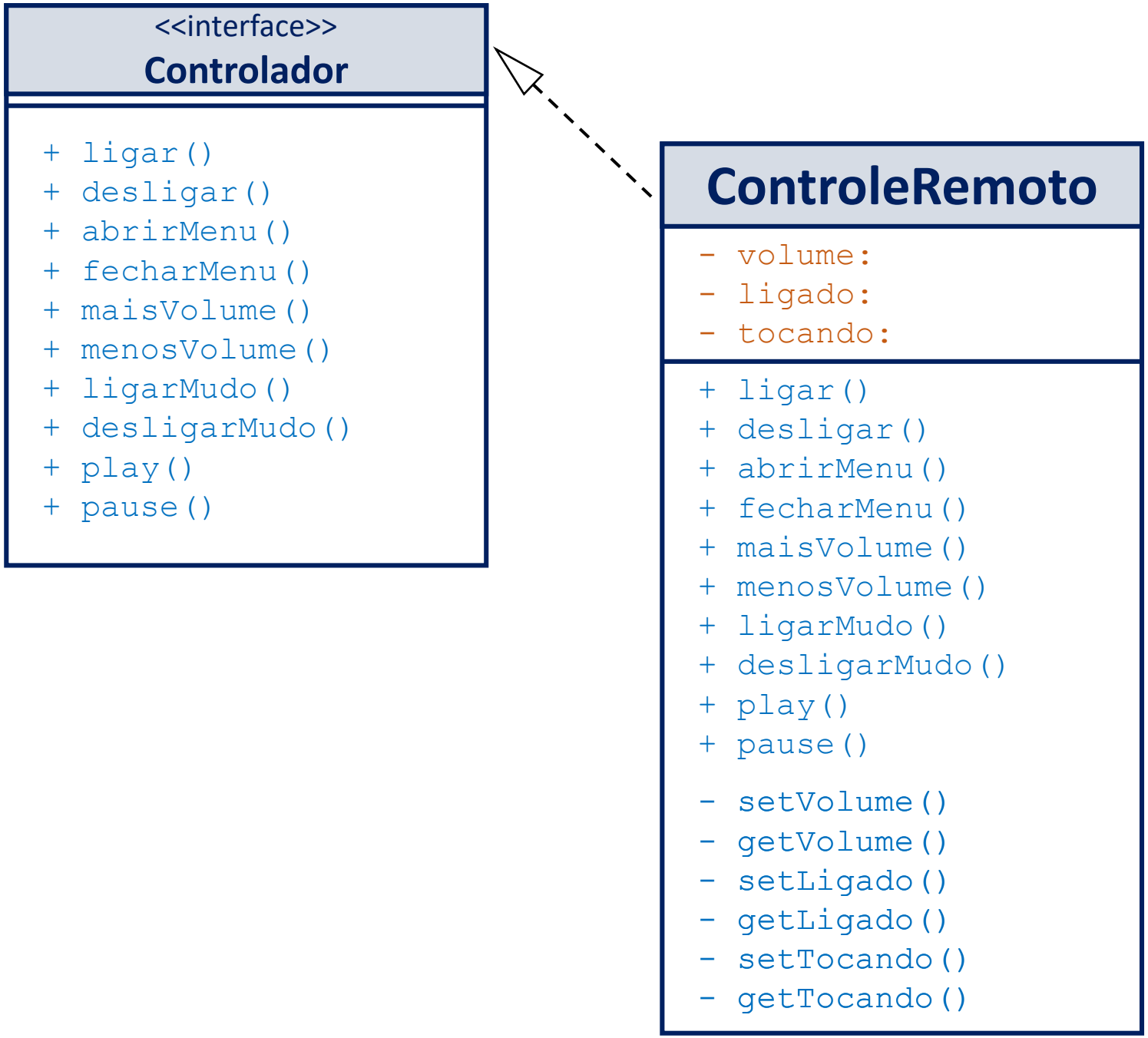
Conceito de Encapsulamento

Implementando o encapsulamento

Agora que temos a nossa Classe ***ControleRemoto*** com seus atributos definidos, como fazer com que ela use o ***Controlador*** como interface?



Fazemos isso utilizando uma seta pontilhada conforme indicado na figura.



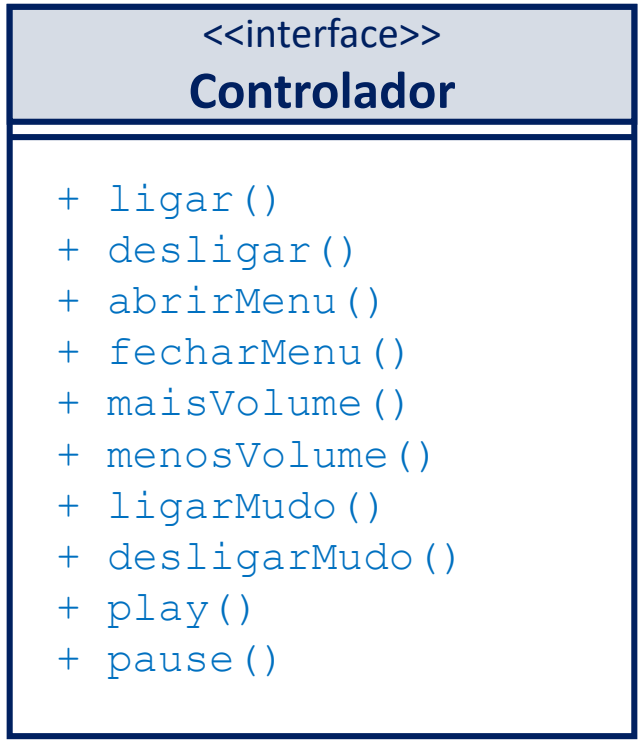
Além de todos os métodos abstratos provenientes do ***Controlador***, precisamos também implementar todos os métodos especiais (***getters*** e ***setters***) da classe.

Conceito de Encapsulamento

Implementando o encapsulamento

Implementado a interface:

A interface é um arquivo em separado onde definiremos os métodos abstratos. Veja a figura abaixo.




```
interface Controlador
  //Métodos abstratos
  publico abstrato Metodo ligar()
  publico abstrato Metodo desligar()
  publico abstrato Metodo abrirMenu()
  publico abstrato Metodo fecharMenu()
  publico abstrato Metodo maisVolume()
  publico abstrato Metodo menosVolume()
  publico abstrato Metodo ligarMudo()
  publico abstrato Metodo desligarMudo()
  publico abstrato Metodo play()
  publico abstrato Metodo pause()
FimInterface
```

Conceito de Encapsulamento

Implementando o encapsulamento

Implementado a Classe:

A implementação da classe já sabemos como é feita, pois, segue as orientações dadas em aulas anteriores.

```
classe ControleRemoto
  //Atributos
  privado inteiro volume
  privado logico ligado
  privado logico tocando
  //Construtor
  publico Metodo Construtor()
        volume = 50
        ligado = falso
        tocando = falso
  FimMetodo
```

ControleRemoto

- volume:
- ligado:
- tocando:

+ ligar()
+ desligar()
+ abrirMenu()
+ fecharMenu()
+ maisVolume()
+ menosVolume()
+ ligarMudo()
+ desligarMudo()
+ play()
+ pause()

- setVolume()
- getVolume()
- setLigado()
- getLigado()
- setTocando()
- getTocando()

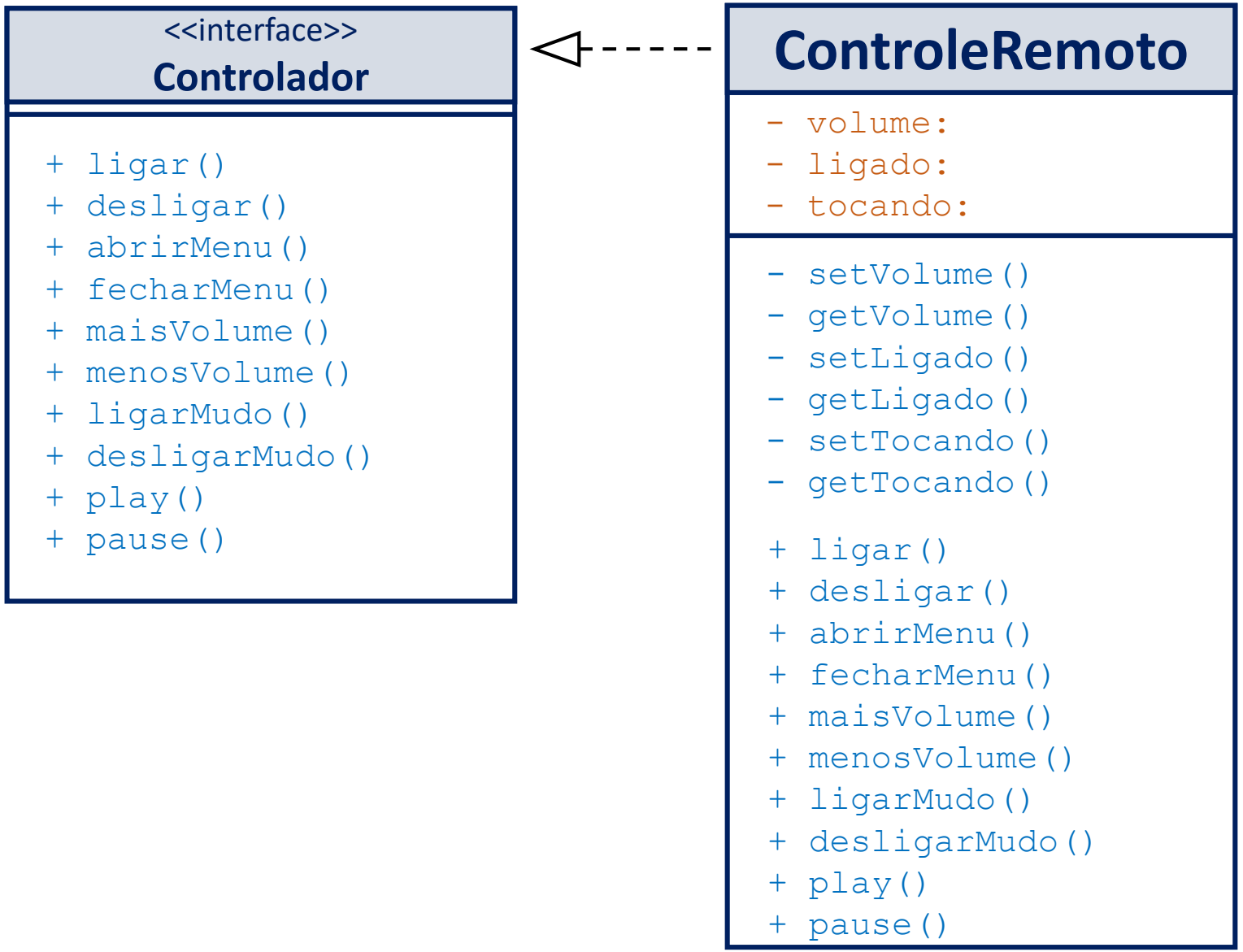
```
privado Metodo getVolume()
    retorne volume
FimMetodo
privado Metodo getLigado()
    retorne ligado
FimMetodo
privado Metodo getTocando()
    retorne tocando
FimMetodo

privado Metodo setVolume(v: inteiro)
    volume = v
FimMetodo
privado Metodo setLigado(l: lógico)
    ligado = l
FimMetodo
privado Metodo setTocando(t: lógico)
    tocando = t
FimMetodo
```

ControleRemoto

- volume:
- ligado:
- tocando:

+ ligar()
+ desligar()
+ abrirMenu()
+ fecharMenu()
+ maisVolume()
+ menosVolume()
+ ligarMudo()
+ desligarMudo()
+ play()
+ pause()
+ setVolume()
- getVolume()
- setLigado()
- getLigado()
- setTocando()
- getTocando()

Agora que já implementamos parte da Classe (atributos e métodos especiais), como fazer para implementar os métodos públicos da interface?



Conceito de Encapsulamento

Implementando o encapsulamento

Implementado a Classe:

Utilizaremos a seguinte sintaxe na construção da classe.

```
classe ControleRemoto implementa Controlador
  //Sobrescrevendo Métodos
  publico Metodo ligar()
    (...)
  FimMetodo
  publico Metodo desligar()
 (...)
  FimMetodo
  publico Metodo abrirMenu()
 (...)
  FimMetodo
  publico Metodo fecharMenu()
 (...)
  FimMetodo
  publico Metodo maisVolume()
 (...)
  FimMetodo
```

ControleRemoto

```
- volume:
- ligado:
- tocando:

+ ligar()
+ desligar()
+ abrirMenu()
+ fecharMenu()
+ maisVolume()
+ menosVolume()
+ ligarMudo()
+ desligarMudo()
+ play()
+ pause()

- setVolume()
- getVolume()
- setLigado()
- getLigado()
- setTocando()
- getTocando()
```

Conceito de Encapsulamento

Implementando o encapsulamento

Implementado a Classe:

Utilizaremos a seguinte sintaxe na construção da classe.

```
    publico Metodo menosVolume()
    (...)
    FimMetodo
    publico Metodo ligarMudo()
    (...)
    FimMetodo
    publico Metodo desligarMudo()
    (...)
    FimMetodo
    publico Metodo play()
    (...)
    FimMetodo
    publico Metodo pause()
    (...)
    FimMetodo
  FimClasse
```

ControleRemoto

- volume:
- ligado:
- tocando:

+ ligar()
+ desligar()
+ abrirMenu()
+ fecharMenu()
+ maisVolume()
+ menosVolume()
+ ligarMudo()
+ desligarMudo()
+ play()
+ pause()

- setVolume()
- getVolume()
- setLigado()
- getLigado()
- setTocando()
- getTocando()

Pilares da POO

Herança

Conceito de Herança

Herança é um mecanismo da Orientação a Objetos que permite criar novas classes a partir de classes já existentes, aproveitando-se das características existentes na classe a ser estendida. Este mecanismo é muito interessante pois promove um grande reuso e reaproveitamento de código existente.

Com a herança é possível criar classes derivadas (subclasses) a partir de classes bases (superclasses). As subclasses são mais especializadas do que as suas superclasses, mais genéricas.

Uma das grandes vantagens de usar o recurso da herança é na reutilização do código. Esse reaproveitamento pode ser acionado quando se identifica que o atributo ou método de uma classe será igual para as outras. Para efetuar uma herança de uma classe é utilizada a palavra reservada chamada ***extends***.

Conceito de Herança

Imagine que dentro de uma organização empresarial, o sistema de RH tenha que trabalhar com os diferentes níveis hierárquicos da empresa, desde o funcionário de baixo escalão até o seu presidente.

Todos são funcionários da empresa, porém cada um com um cargo diferente. Mesmo a secretária, o pessoal da limpeza, o diretor e o presidente possuem um número de identificação, além de salário e outras características em comum.

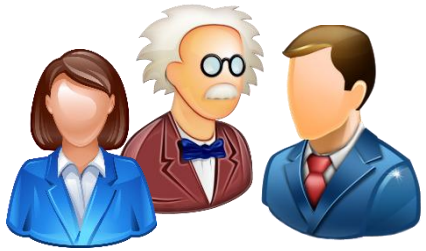
Classe: Funcionário

Classe: Diretor

Classe: Tecnico

Herança

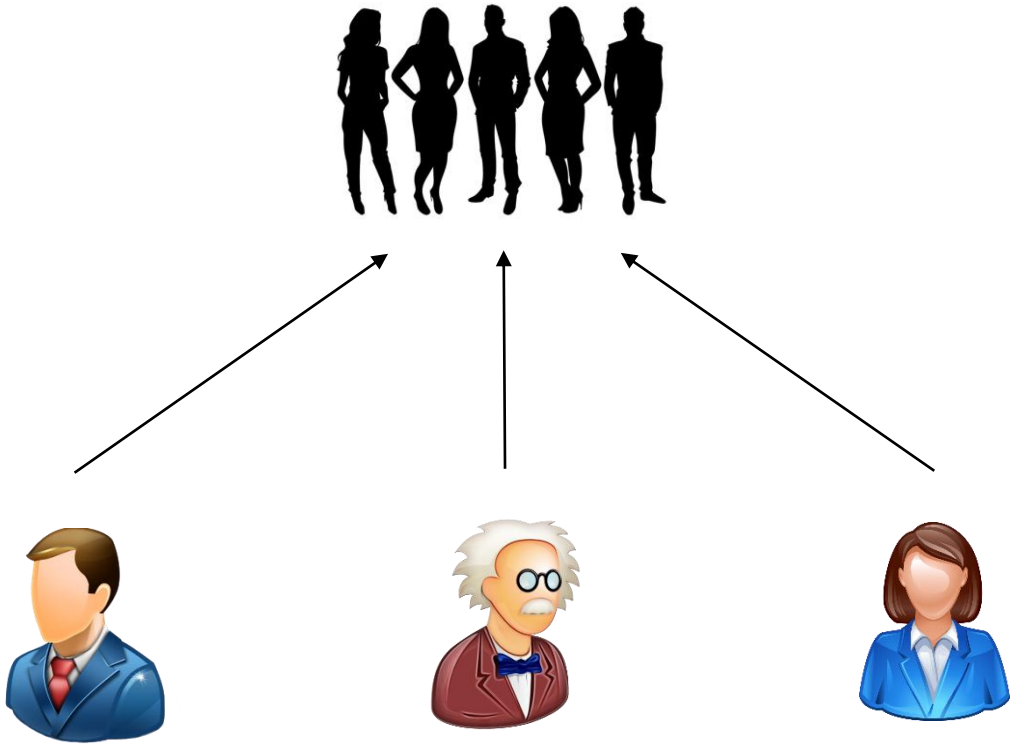
Vejamos o exemplo a seguir:



Aluno
- nome - idade - sexo - matricula - curso
+ fazerAniversario() + cancelarMatricula()

Professor
- nome - idade - sexo - especialidade - salario
+ fazerAniversario() + receberAumento()

Funcionario
- nome - idade - sexo - setor - trabalhando
+ fazerAniversario() + mudarTrabalho()



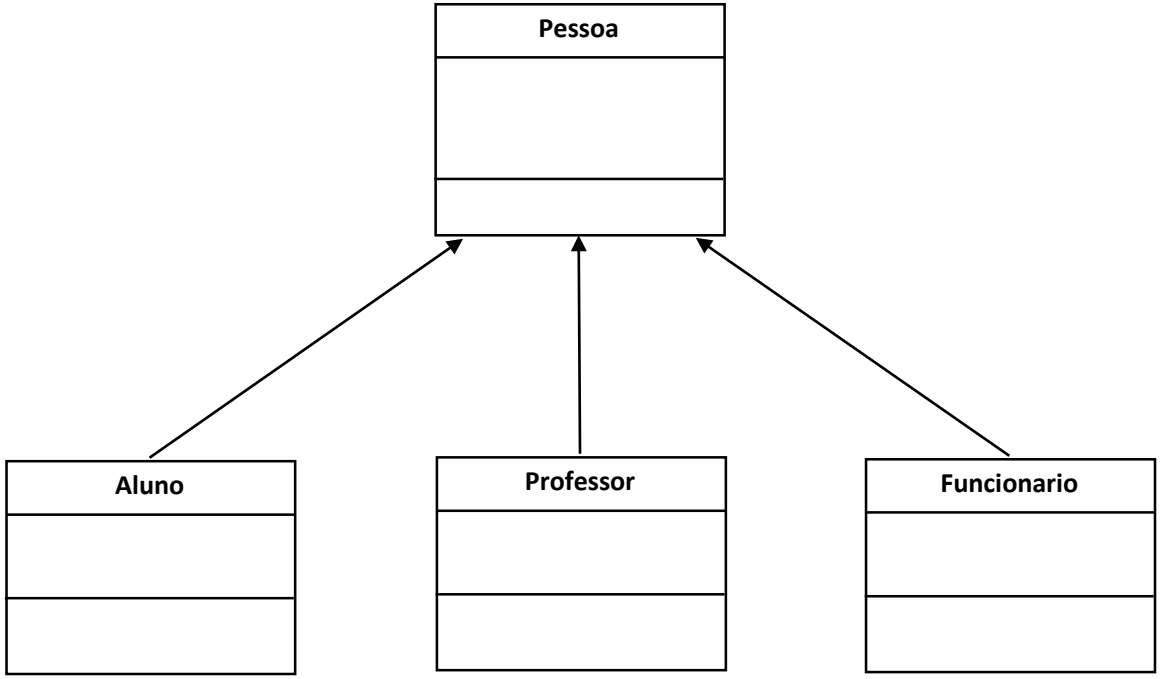
Herança

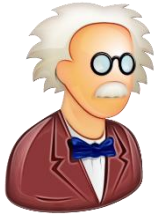
Vejamos o exemplo a seguir:

Aluno
- nome - idade - sexo - matricula - curso
+ fazerAniversario() + cancelarMatricula()

Professor
- nome - idade - sexo - especialidade - salario
+ fazerAniversario() + receberAumento()

Funcionario
- nome - idade - sexo - setor - trabalhando
+ fazerAniversario() + mudarTrabalho()





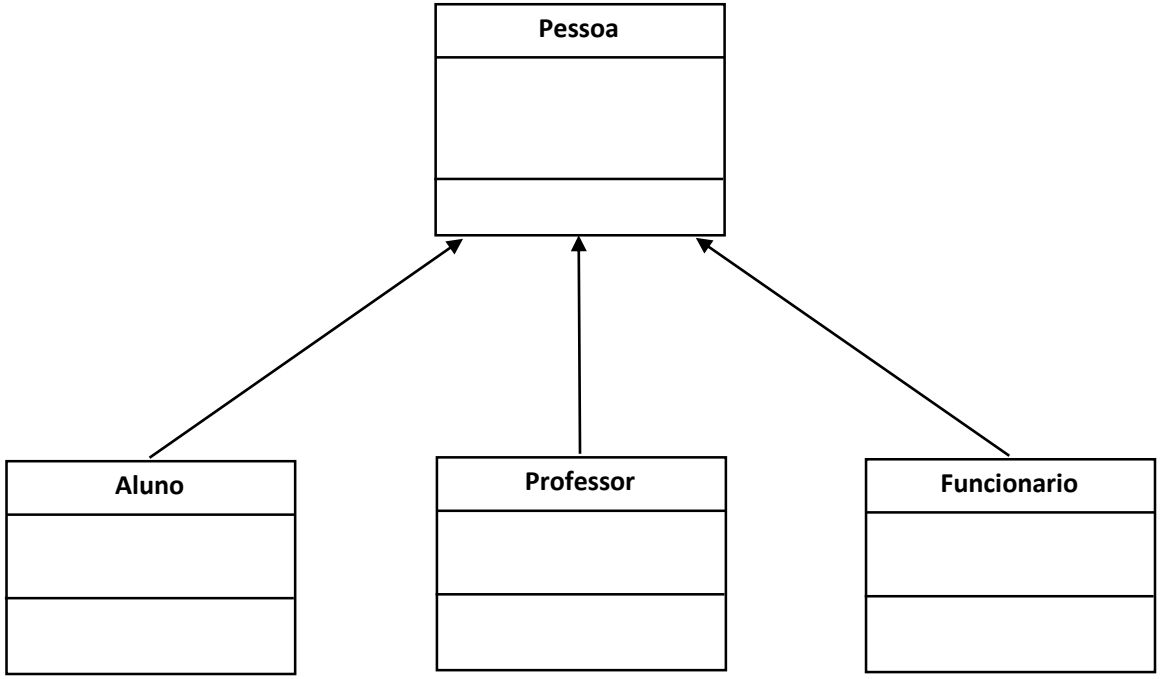
Herança

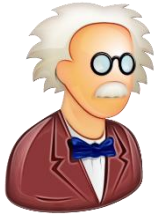
Vejamos o exemplo a seguir:

Aluno
- nome - idade - sexo - matricula - curso
+ fazerAniversario() + cancelarMatricula()

Professor
- nome - idade - sexo - especialidade - salario
+ fazerAniversario() + receberAumento()

Funcionario
- nome - idade - sexo - setor - trabalhando
+ fazerAniversario() + mudarTrabalho()





Herança

Vejamos o exemplo a seguir:

Aluno
- nome - idade - sexo - matricula - curso
+ fazerAniversario() + cancelarMatricula()

Professor
- nome - idade - sexo - especialidade - salario
+ fazerAniversario() + receberAumento()

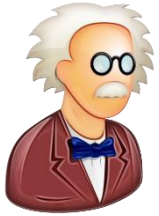
Funcionario
- nome - idade - sexo - setor - trabalhando
+ fazerAniversario() + mudarTrabalho()

Pessoa
- nome - idade - sexo
+ fazerAniversario()

Aluno
- matricula - curso
+ fazerMatricula()

Professor
- especialidade - salario
+ receberAumento()

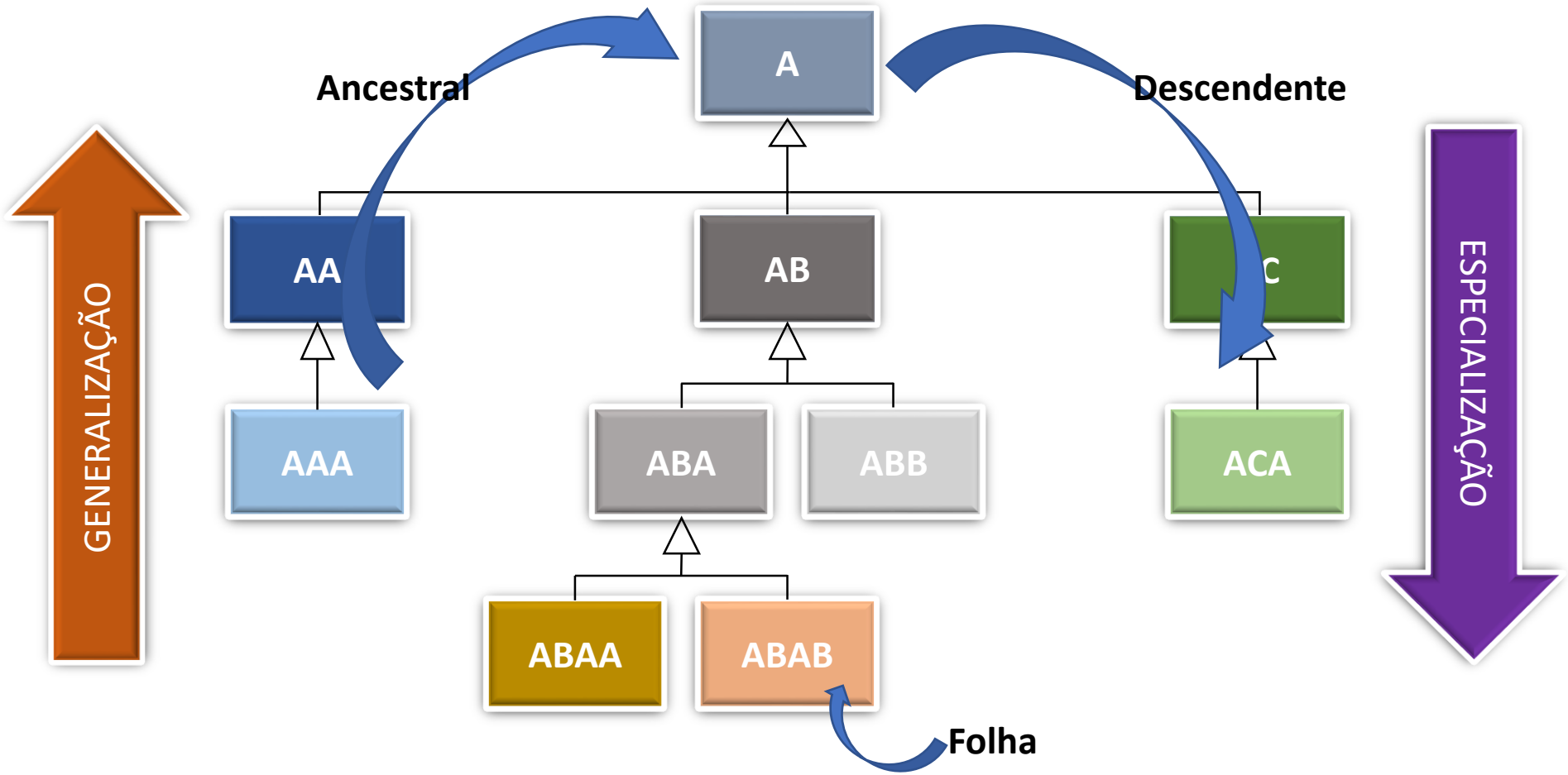
Funcionario
- setor - trabalhando
+ mudarTrabalho()



Conceito de Herança

Árvores hierárquicas em POO

Na Programação Orientada a Objetos o conceito de herança e a hierarquia entre classes traz vários conceitos importantes que precisamos aprender, veja o exemplo a seguir:

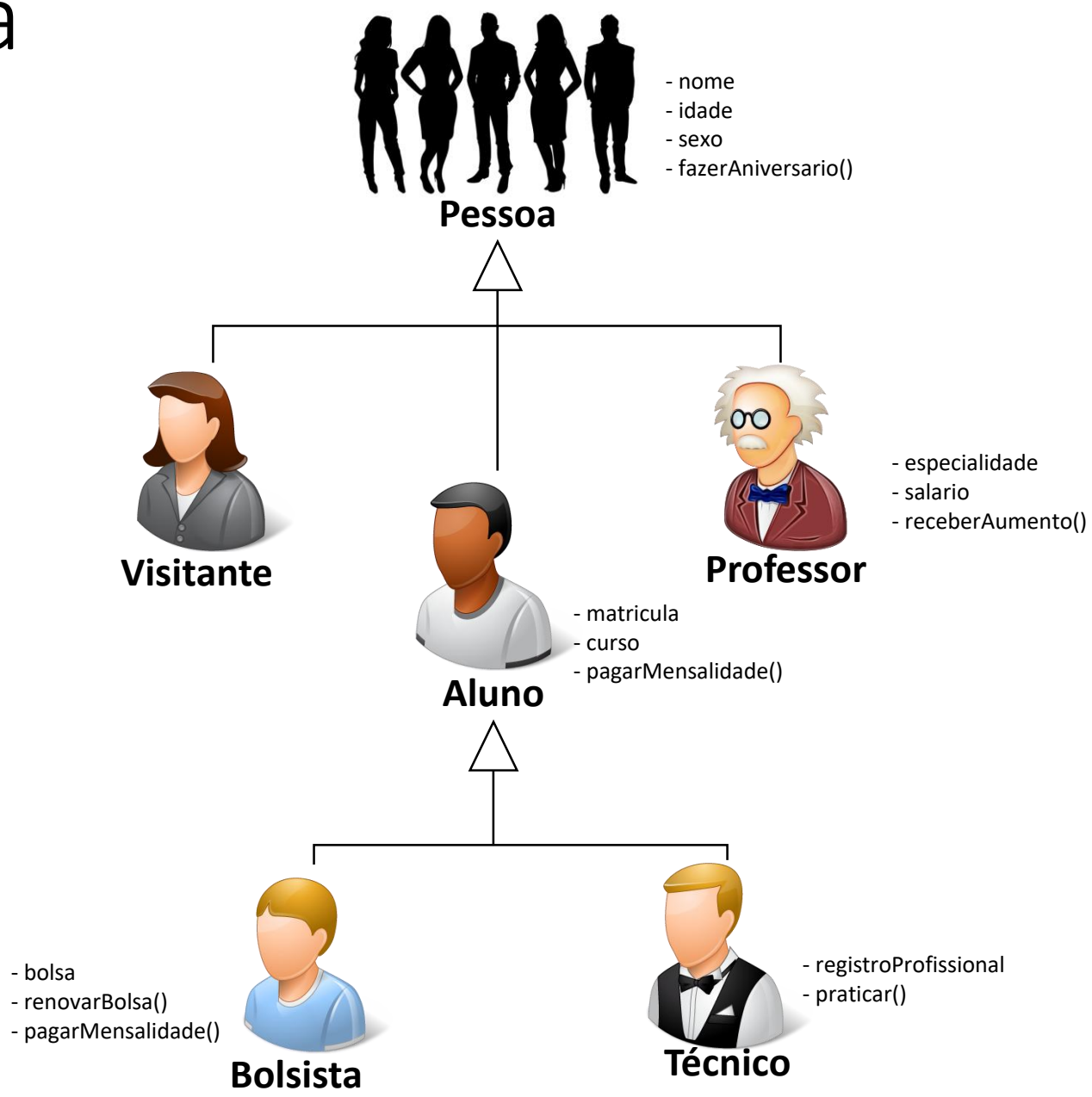


Conceito de Herança

Tipos de Herança

1 **Herança de Implementação**

2 **Herança para Diferença**



Conceito de Herança

Conceito de Abstrato e Final

Na Programação Orientada a Objetos podemos ter classes que não podem ser instanciadas e também métodos que que não podem ser implementados. A essas classes e métodos denominamos de Abstratos.

Assim uma classe abstrata somente pode gerar outras subclasses, como por exemplo, a classe ***Pessoa*** que gerou a classe ***Aluno***, ***Professor*** e ***Funcionário***. Já um método abstrato tem como característica somente a declaração, mas não a implementação, como por exemplo, o que aconteceu na construção da Interface.

Já o conceito de classe final representa uma classe que não pode ter filhos, seria uma classe tipo folha na árvore hierárquica, e um método final é aquele que não pode ser sobrescrito, obrigatoriamente apenas herdado.

Conceito de Herança

Conceito de Abstrato e Final

Classe Abstrata

Não pode ser instanciada. Só pode servir como progenitora.

Método Abstrato

Declarado, mas não implementado na progenitora.

Classe Final

Não pode ser herdada por outra classe.
Obrigatoriamente é folha

Método Final

Não pode ser sobrescrito pelas suas subclasses.
Obrigatoriamente herdado.

Pilares da POO

Polimorfismo

Conceito de Polimorfismo

Formalmente polimorfismo quer dizer “várias formas”. E no caso da Orientação a Objeto, o polimorfismo denota uma situação na qual um objeto pode se comportar de maneiras diferentes ao receber uma mensagem, dependendo do seu tipo de criação.

O polimorfismo é alcançado com auxílio do uso de herança nas classes e a reescrita (overriding) de métodos das superclasses nas suas subclasses.

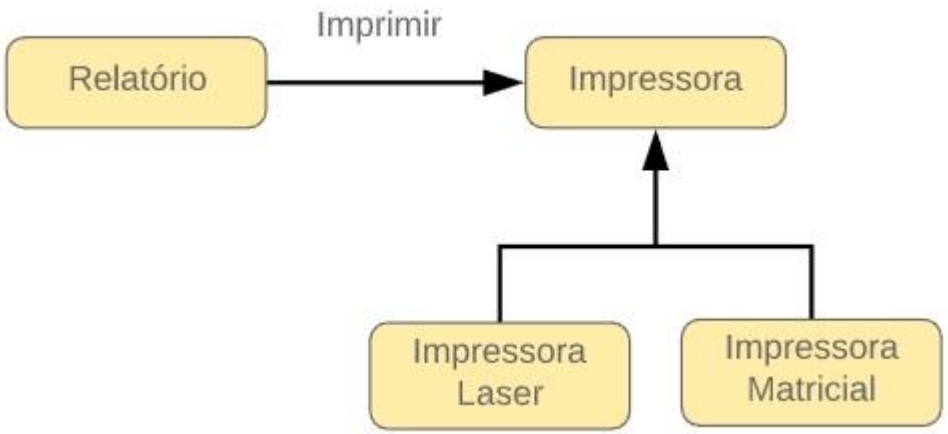
Conceito de Polimorfismo

Duas subclasses de uma mesma classe podem ter implementações completamente diferentes de um mesmo método, o que leva os objetos a se comportarem de forma diferente, dependendo do seu tipo (classe).

Exemplificando, podemos imaginar um programa que faça a impressão de um relatório, por meio de uma classe chamada Impressora, que é uma interface de acesso às funcionalidades da impressora usada, por meio de um driver fornecido pelo fabricante.

Conceito de Polimorfismo

Uma impressora a laser tem um mecanismo de impressão totalmente diferente de uma impressora a jato de tinta ou matricial, mas isso não importa para o programa.



Ele manda uma simples mensagem de imprimir para a impressora, e o modo como a impressora imprime no papel varia de acordo com o tipo de impressora usada, ou seja, a impressão se dá de formas diferentes para a mesma mensagem de imprimir.

Algumas linguagens promovem o polimorfismo principalmente através do uso de classes abstratas e interfaces.